Learning Objectives

1. Classes in Python.
2. Functions in Python
3. Graphs in Python (coded as Dictionary)

1. Classes in Python

If you are a beginner in python programming, you must be knowing about primitive data types like integers, floating point numbers, strings and complex numbers. Also, till now we have built data structures like python dictionary, list, tuple and set. Now we will learn to create complex data types to store information about real world objects using classes in python.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
    def myfunction(self):
        print("Hello my name is " + self.name)
        print("Hello my age is " + self.age)
```

```
p1 = Person("Ali", 25)
p1.myfunction()
```

Class: a blueprint for how to build a certain type of object. The Person class (shown above) describes the behavior and data that all Person objects have.
Instance: a particular occurrence of a class.
Attribute or field: a variable that belongs to the class. Think of an attribute as a quality of the object: Person have name and age, so we have given **Person class** self.name and self.age attributes.
Method: Methods are just like normal functions, except that they are tied to an instance or a class. Think of a method as a "verb" of the class: cars can drive and also pop their tires. We call methods using dot notation.
Constructor: As with data abstraction, constructors describe how to build an instance of the class. Most classes have a constructor. In Python, the constructor of the class defined as __init__.
self: in Python, self is the first parameter for many methods (in this class, we will only use methods whose first parameter is self). When a method is called, self is bound to an instance of the class. Notice that the `myfunction` method takes in self as an argument, but it looks like we

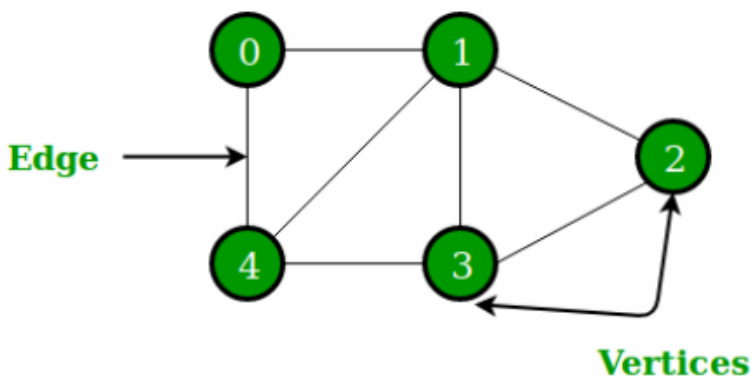
didn't pass one in! This is because the dot notation implicitly passes in person as self for us.

2. Graph

A graph is collection of two sets V and E where V is a finite non-empty set of vertices and E is a finite non-empty set of edges.

- Vertices are nothing but the nodes in the graph.
- Two adjacent vertices are joined by edges.
- Any graph is denoted as G = {V, E}.

For example:

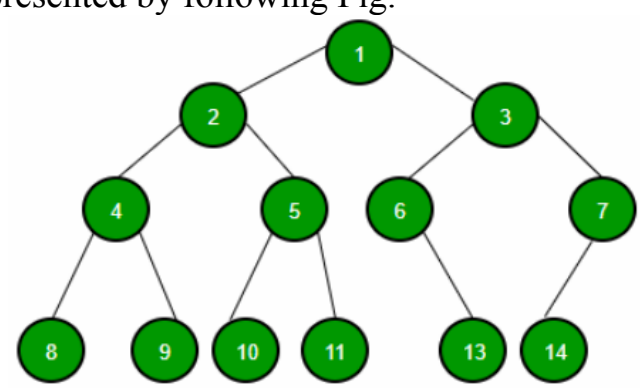


G = {{V1, V2, V3, V4, V5, V6}, {E1, E2, E3, E4, E5, E6, E7}}

Whereas, a tree is a finite set of one or more nodes such that:

- There is a specially designated node called root.
- The remaining nodes are partitioned into n>=0 disjoint sets T1, T2, T3, …, Tn where T1, T2, T3, …, Tn is called the subtrees of the root.

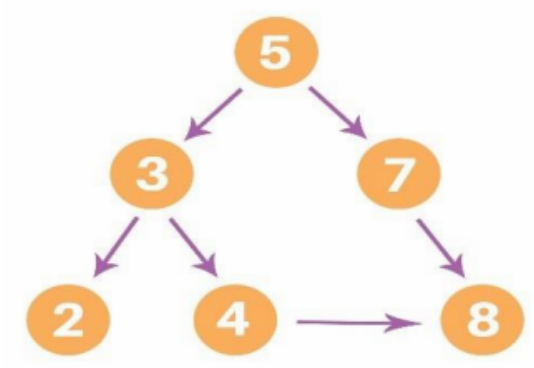
The concept of tree is represented by following Fig.



There are different types of blind or uninformed searches that can be used to solve a certain problem.

Commonly used search techniques are:

1. Breadth First Search
2. Depth First Search

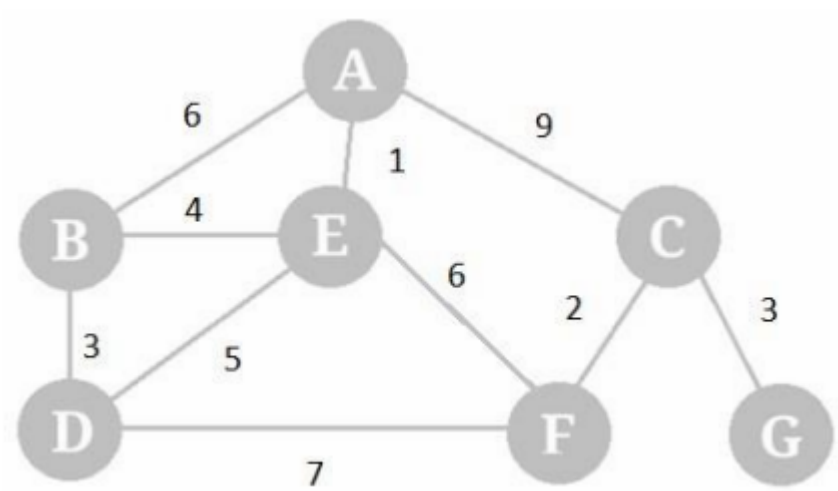



```python
graph1 = {
    5: [3, 7],
    3: [2, 4],
    7: [8],
    2: [],
    4: [8],
    8: []
}

for node in graph1:
    print(str(node) + " --> " + str(graph1[node]))
```

```
5 --> [3, 7]
3 --> [2, 4]
7 --> [8]
2 --> []
4 --> [8]
8 --> []
```

3. Weighted Graph



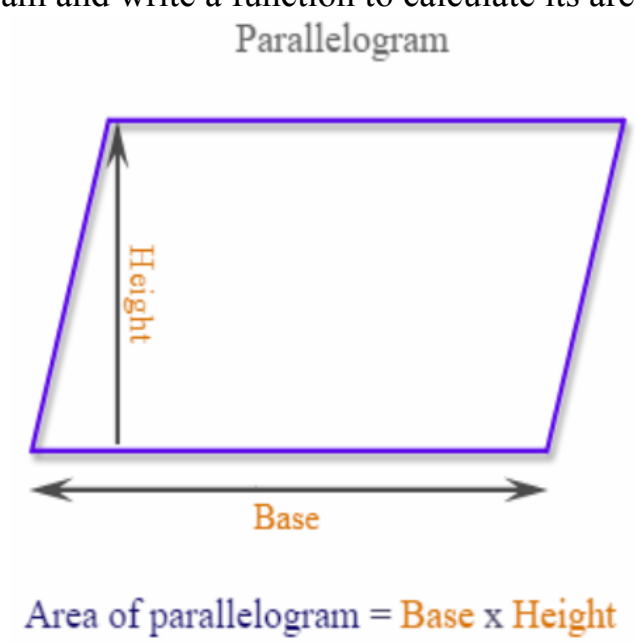
```python
graph2 = {
    'A': {'B': 6, 'E': 1, 'C': 9},
    'B': {'A': 6, 'E': 4, 'D': 3},
    'C': {'A': 9, 'F': 2, 'G': 3},
    'D': {'B': 3, 'E': 5, 'F': 7},
    'E': {'A': 1, 'B': 4, 'D': 5, 'F': 6},
    'F': {'E': 6, 'D': 7, 'C': 2},
    'G': {'C': 3}
}

for node in graph2:
    print(node + " --> " + str(graph2[node]))
```

```
A --> {'B': 6, 'E': 1, 'C': 9}
B --> {'A': 6, 'E': 4, 'D': 3}
C --> {'A': 9, 'F': 2, 'G': 3}
D --> {'B': 3, 'E': 5, 'F': 7}
E --> {'A': 1, 'B': 4, 'D': 5, 'F': 6}
F --> {'E': 6, 'D': 7, 'C': 2}
G --> {'C': 3}
```

Lab Tasks

1. Make a class parallelogram and write a function to calculate its area.



Lab # 03 Marks distribution

	ER1	ER6	ER8
Task	3 points	3 points	4 points

Lab # 03 Rubric Evaluation Guideline:

#	Qualities & Criteria	0 < Poor <= 1	1 < Satisfactory <= 2	2 < Excellent <=3
ER1	**Task Completion**	Minimal or no program functionality was achieved.	Some tasks were completed, but the program has errors or incomplete functionalities.	All tasks were completed, and the program runs without errors.
#	**Qualities & Criteria**	**0 < Poor <= 1**	**1 < Satisfactory <= 2**	**2 < Excellent <=3**
ER6	**Program Output**	Output is inaccurate or poorly presented.	Output is mostly accurate but may lack labels, captions, or formatting.	Output is clear, accurate, and well presented with labels, captions, and proper formatting.
#	**Qualities & Criteria**	**0 < Poor <= 1**	**1.5 < Satisfactory <= 2.5**	**2.5 < Excellent <= 4**
ER8	**Question & Answer**	Answers some questions but not confidently or based on lab task knowledge.	Answers most questions confidently and based on lab task knowledge.	Answers all questions confidently and demonstrates a deep understanding of the given lab task.